

Module TH-9: The machine learning paradigm

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

After this module you can:

- ▶ Explain what changes when a program no longer holds the rules and a model holds them instead.
- ▶ Name the four parts of a learned model: features, labels, model, and loss, and say what each one is.
- ▶ Decide if a problem needs written rules or a learned model, and name the reason for the decision.

A device must tell three movements apart: waving, flipping, and idling. One team writes the program. Another team records the movements and lets a program find the pattern.

- ▶ Both teams end with a program that runs on the device.
- ▶ The rules live in a different place. In the first team a programmer wrote every rule. In the second team the data holds the rules.

Key point: The question is not whether to use machine learning. The question is who writes the rules.



The programmer writes the rules first. The data arrives later, and the program applies the rules to it.

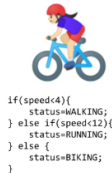
Key point: The rules are an input of the program.

A game needs a rule for a collision, because the physics of the game is known and it never changes.

- ▶ The states are few, and a person can list all of them.
- ▶ The correct answer is exact. No answer is close enough.
- ▶ A wrong answer is visible at once. The game looks broken.

A thermostat with a fixed threshold is the same case: the rule is one line, and its test is a measurement of a number.

Where written rules break



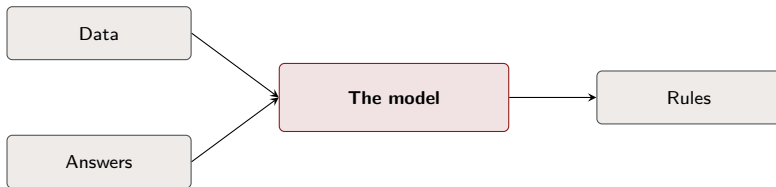
The rule for walking is one test. Biking needs a second test. A movement that nobody wrote down needs a new branch, and no end is in sight.

Source: Figure adopted from Volume I, chapter 5.

Movements named	Branches in the code	Lines to maintain
2	2	about 10
3	3	about 15
4	4	about 20

The count of branches follows the count of movements, and the count of movements follows the people who use the device. The code is correct at every step and wrong at the end, because the last movement is missing.

Source: Illustration of this course, after the growth of the rule count in Volume I, chapter 5. Nobody measured these line counts.



The program no longer holds the rules. It finds them, and it holds the result.

Key point: The rules are the output of the program.

$$\text{answer} = f(\text{input}; \theta)$$

- ▶ f is the shape of the computation. It never changes after the training.
- ▶ θ holds the parameters: the weights and the biases. The training moves them.
- ▶ A neuron of the simplest kind has two parameters. Its output is `weight * input + bias`, so for the six points of the next frames the weight becomes 2 and the bias becomes -1 .

The four parts of a learned model

Part	What it is	Who gives it
Features	The input as a vector of numbers	The sensors, the code
Labels	The right answer for each example	A person, most of the time
Model	The function f and its parameters θ	The training
Loss	One number for the distance to the labels	The choice of the team

Three of the four parts come from the team. The fourth is what the training produces.

A feature is a number that describes one thing about the input.

- ▶ An accelerometer of a waving hand gives three numbers at each sample: the acceleration on the three axes. The sample rate adds the time.
- ▶ A photograph is already a vector of numbers: 784 grey values for a 28 by 28 image.
- ▶ The engineer chooses the features. A feature that the sensor does not measure needs another sensor or another code.

The features of the classic example are the 784 pixel values of one handwritten digit.

Source: Dimensions of the running example of Volume I, chapter 5.

A label is the answer that belongs to one example.

- ▶ For the six points of the regression, the label of each x is its y .
- ▶ For a movement, the label is the name of the movement.
- ▶ The label of a digit is its number.
- ▶ Getting the labels is the slow part of the work. A radiologist reads each image, and two readers disagree on the border cases.

Key point: The model is only as good as the labels.

Take six points. The real values are $y = \{-3, -1, 1, 3, 5, 7\}$ and a guess is $\hat{y} = 3x - 1$.

x	-1	0	1	2	3	4
$\hat{y}$	-4	-1	2	5	8	11
$\hat{y} - y$	-1	0	1	2	3	4

The training needs one number, not six. The root mean square of the six differences is $\sqrt{31} = 5.57$.

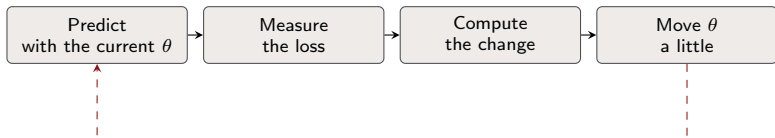
The loss guides every step of the training

The training does two things again and again: it predicts, then it measures.

- ▶ A guess with a loss of 5.57 is worse than a guess with a loss of 0.5. The number gives the order.
- ▶ The training moves the parameters in the direction that lowers the loss.
- ▶ A guess that fits the six points exactly, $y = 2x - 1$, has a loss of 0.

Loss: one number for the distance between the answers of the model and the labels.

Training is a loop



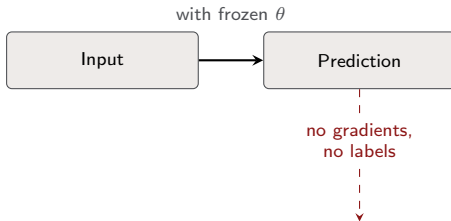
again, until the loss stops falling

A network starts with random parameters, so its first prediction is close to a guess of ten classes with no information. Each turn of the loop makes the next prediction a little better.

Show the model examples with their labels, and move the parameters so that the loss falls.

- ▶ The examples come from a dataset. The dataset of the classic example holds 60 000 training images and 10 000 validation images.
- ▶ The moves are small. A step that is too large passes over the answer and the loss rises again.
- ▶ The loop ends when the loss on unseen examples stops falling.

Inference is the forward pass alone



Inference runs the forward pass and stops. It keeps no gradients and no labels, so it needs far less memory than the training.

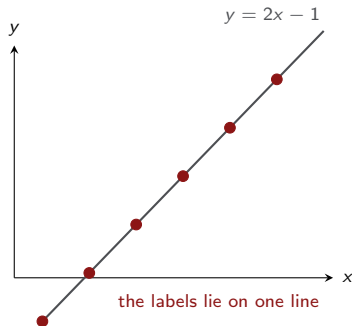
Key point: The training needs a big machine. The inference runs on the device.

Training and inference are two different jobs

	Training	Inference
Passes	Forward and backward	Forward only
Parameters	Move at every step	Frozen
Memory	Activations, gradients, optimizer	The weights and one input
Data	Labelled, on a server	One input, on the device
Latency	Seconds to days	Milliseconds

The two jobs use the same weights, and they need different hardware. A microcontroller runs the second one only.

Regression predicts a number



The answer of a regression is a number, so the loss can be the distance between two numbers. A temperature, a count, and a price are regressions.

	Regression	Classification
The answer is	One number	One of k classes
The loss is	The distance	$-y \log \hat{y}$
The output is	A value	k probabilities
An example	A temperature	The digit in an image

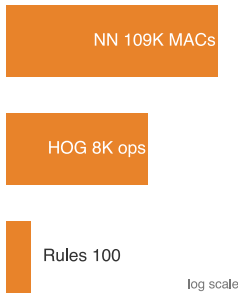
The distance of the regression has no meaning for a class. A classification therefore uses another loss, and the loss of the courseware for a digit is $-\sum_i y_i \log \hat{y}_i$.

A class becomes a vector of zeros

A classification has no order between the classes, so the label of the digit 3 is not a 3.

Digit	Label vector
0	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
3	[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
9	[0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Exactly one entry is 1. The model returns ten probabilities that sum to 1, so the highest entry names the class.

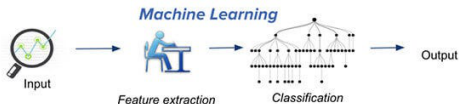


- ▶ The written rules of the running example need about 100 comparisons.
- ▶ The learned features need about 8 000 operations.
- ▶ The neural network needs 109 184 multiply-accumulate operations.

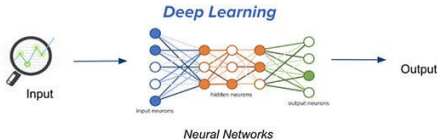
The change is not free. A model buys its rules with arithmetic.

Source: Figure adopted from Volume I, chapter 5, and the operation counts of its running example.

There is a step between the two paradigms



Traditional machine learning uses hand-crafted features, which is tedious and costly to develop.



Deep learning learns hierarchical representation from the data itself, and scales with more data.

The upper row asks a person to design the features. The lower row lets the training design them. The middle option is the most common today on a small device.

Source: Figure adopted from Volume I, chapter 5.

Which tool does the problem need

Use written rules when	Use a learned model when
The states can be listed	The states cannot be listed
The right answer is exact	The right answer is close enough
A wrong answer shows at once	A wrong answer stays hidden
The rules change often	The rules change with the data

Most systems hold both. A thermostat reads a number with a written rule and decides "this person fell" with a model.

Key point: Ask what the wrong answer costs. That question has no rule.

- ▶ **A model replaces the program.** It replaces the rules inside the program. The sensor, the timing, and the memory budget stay.
- ▶ **More data removes the work.** More data removes the rule count. It does not remove the labels, the hardware, or the test on unseen data.
- ▶ **A learned model is always the smaller program.** The model of the running example needs about 1 100 times the arithmetic of the written rules, so the budget moves to the arithmetic.

Key point: Learning moves the difficulty. It does not remove it.

1. Written rules and a learned model give an answer. The difference is where the rules live.
2. Four parts: features, labels, model, and loss.
3. The training is a loop of predict, measure, and move. The inference is the forward pass alone.
4. Regression answers with a number. Classification answers with a class.
5. The decision rests on what a wrong answer costs.

A team has three problems.

1. A smart lock must open when the six digits of a code match the six digits that are stored.
2. A watch must count the steps of a person from the readings of its accelerometer.
3. A camera must find the three defects on the printed circuit boards that leave the factory.

For each problem, decide if written rules or a learned model is the correct tool, and give one reason.

1. **Written rules.** The correct answer is exact, and a wrong answer is visible at once: the lock does not open. The states can be listed, because a code is six digits.
2. **A learned model.** The correct pattern of the readings is not known in advance, and it changes with the person and with the position of the watch. A wrong step count stays hidden, so the cost of the mistake is low and the number of cases is large.
3. **A learned model.** The position and the light of the defects vary, and nobody can list them. A wrong answer on a board that is good is the normal case, so the model must be measured on real boards.

The rule that separates the three cases is the cost of the wrong answer. A lock that stays shut costs little, so a simple rule is enough. A board that ships with a defect costs a customer, so the tool must find the rare case.

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *HarvardX TinyML courseware* by the TinyMLx team. github.com/tinyMLx/courseware, CC BY-NC-SA 4.0.